



Title: Coding Problem and Details

```
```python
import numpy as np
import pandas as pd
Goal: predict a binary target (have_insurance: 0/1) from features:
- age (numeric)
- income (numeric)
- education (categorical)
#
We'll:
1) Load CSV
2) Select and preprocess features (scale income, one-hot encode education)
3) Add a bias (intercept) term
4) Train logistic regression with gradient descent
5) Print loss during training and learned weights
6) Compute training accuracy
=====
---- 1) Load data ----
Reads a CSV file into a DataFrame where each column is a variable/feature.
df = pd.read_csv("data.csv")

---- 2) Build feature matrix X ----
We pick only the feature columns we want for the model.
.copy() ensures we modify a new DataFrame, not a view into df (avoids
SettingWithCopy issues).
X = df[["age", "income", "education"]].copy()

---- 3) Feature scaling (income) ----
Income values may be large (e.g., 50000) compared to age (e.g., 35).
Gradient descent usually converges faster when feature magnitudes are
similar.
Dividing income by 1000 rescales it (e.g., 50000 -> 50).
X["income"] = X["income"] / 1000.0

---- 4) One-hot encode the categorical column ----
Logistic regression needs numeric inputs. 'education' is categorical (e.g.,
"HS", "College").
get_dummies converts it into multiple 0/1 columns (one per category).
#
drop_first=True drops the first category to avoid perfect multicollinearity
(a.k.a. the "dummy variable trap") where columns sum to 1 and become
redundant.
X = pd.get_dummies(X, columns=["education"], drop_first=True)

---- 5) Prepare target vector y ----
have_insurance should be 0 or 1.
```

---

```
.values gives a NumPy array. reshape(-1, 1) makes it a column vector of
shape (n, 1),
which aligns nicely with matrix multiplication later.
y = df["have_insurance"].values.reshape(-1, 1)

---- 6) Convert X to NumPy float matrix ----
After one-hot encoding, X is still a DataFrame. For fast linear algebra, we
convert it to NumPy.
astype(float) ensures all values are numeric floats.
X = X.values.astype(float)

n = number of samples (rows), d = number of features (columns) after
preprocessing
n, d = X.shape
---- 7) Add bias (intercept) column ----
Logistic regression usually has an intercept term b.
Instead of keeping b separately, we append a column of ones and treat bias
as another weight.
#
Xb will be shape (n, d+1). The last column is all ones.
Xb = np.hstack([X, np.ones((n, 1))])

---- 8) Initialize weights ----
w is a column vector of shape (d+1, 1).
Starting from zeros is fine for logistic regression (unlike some neural
nets).
w = np.zeros((d + 1, 1))
---- 9) Sigmoid function ----
Logistic regression models:
$p(y=1|x) = \text{sigmoid}(z)$ where $z = x \cdot w$
Sigmoid maps real numbers $\rightarrow (0, 1)$, interpretable as a probability.
def sigmoid(z):
 return 1 / (1 + np.exp(-z))

---- 10) Training hyperparameters ----
lr = 0.05 # learning rate: step size for gradient descent updates
epochs = 2000 # number of passes over the data
=====
Training loop: gradient descent on logistic (cross-entropy) loss
=====
for epoch in range(epochs):
 # ---- Forward pass ----
 # Linear scores (logits): $z = Xb @ w$
 # Shape: (n, d+1) @ (d+1, 1) $\rightarrow$ (n, 1)
 z = Xb @ w
 # Predicted probabilities for class 1
 # p[i] is model's probability that sample i has insurance
 p = sigmoid(z)
 # ---- Compute gradient ----
```

```
For logistic regression with cross-entropy loss, gradient is:
grad = (Xb^T @ (p - y)) / n
#
Intuition:
- (p - y) is the error in probability space.
- Multiplying by Xb^T accumulates how each feature contributes to the
error.
- /n averages gradient across samples.
grad = (Xb.T @ (p - y)) / n
---- Update weights ----
Gradient descent step: w = w - lr * grad
w -= lr * grad
---- Optional: print loss sometimes ----
Loss function (binary cross-entropy / log loss):
L = -mean(y*log(p) + (1-y)*log(1-p))
#
We add eps to avoid log(0) which would produce -inf or NaN due to
floating point limits.
if epoch % 400 == 0:
 eps = 1e-9
 loss = -np.mean(y*np.log(p + eps) + (1 - y)*np.log(1 - p + eps))
 print(f"epoch {epoch:4d} loss={loss:.4f}")
=====
Interpret learned weights
=====
Reconstruct feature names in the SAME order as the dummy-encoding used
earlier.
NOTE: This recomputes get_dummies to get column names; the order should
match
Pandas' get_dummies behavior (typically consistent given the same data).
feature_names = list(
 pd.get_dummies(
 df[["age", "income", "education"]],
 columns=["education"],
 drop_first=True
).columns
)
Append "bias" to match the final column of ones we added to Xb.
feature_names.append("bias")
print("\nLearned weights:")
for name, val in zip(feature_names, w.ravel()):
 # Interpretation:
 # - Positive weight: increases probability of y=1 as that feature
increases (or is present).
 # - Negative weight: decreases probability of y=1.
 # For dummy vars (education_*), weight compares that category vs the
dropped baseline category.
 print(f"{name:20s}: {val:.4f}")
---- 11) Predict (0/1) for the training data ----
Compute probabilities using the trained weights
probs = sigmoid(Xb @ w)
Convert probabilities to hard class labels using a threshold of 0.5
preds = (probs >= 0.5).astype(int)
```

```
Training accuracy: fraction of samples where predicted label equals true label
print("\nTraining accuracy:", (preds.ravel() == y.ravel()).mean())
```
```

Gradient descent is a simple “hill-walking” method for **reducing error**.

The idea

- Your model makes predictions.
- You compute a **loss** (a single number) that measures “how wrong” the model is.
- You want the loss to be as small as possible.

Think of the loss as the **height of a hill**:

- High loss = you’re high up (bad)
- Low loss = you’re down in a valley (good)

How it moves

At the current point (current weights):

1. Look at the **slope** of the loss (which direction goes uphill fastest). That slope is the **gradient**.
2. Move a small step in the **opposite direction** (downhill), because you want less loss.

The update rule

- **weights = weights – (learning_rate × gradient)**

Meaning:

- If a weight is making loss worse, decrease it.
- If increasing a weight would reduce loss, increase it (because subtracting a negative gradient increases it).

What “learning rate” means

- Learning rate = **step size**
- Too big → you might overshoot and bounce around
- Too small → you’ll improve very slowly

Tiny example analogy

If you’re blindfolded on a hill:

- You feel the ground around your feet to sense which way slopes downward.
- You take a small step downhill.
- Repeat until it feels mostly flat (you reached a low point).

In your logistic regression code

- **Loss**: Cross-entropy (how wrong predicted probabilities are)
- **Gradient**: Computed from $(p - y)$ (prediction error) and the inputs X
- Each epoch updates w so predictions get closer to the true labels.



$$\frac{1}{n} X^T (p - y)$$

This means that for each feature, we add up its contribution to the model's errors across all samples.

Here, $p - y$ represents the **signed error** (positive when the prediction is too high, negative when it is too low). Multiplying by X^T weights these errors by how strongly each feature is present.

As a result, the gradient tells each weight to **increase if the feature is common when the model underpredicts, and decrease if it is common when the model overpredicts**, averaged over n samples.

Maths behind code -

https://chatgpt.com/s/t_6987764729f48191b994ee6994ecb4f5